

Phases of a Compiler

CSA1412 - Compiler Design

Guided By:
Dr. Sugumar R

Dept. of CSE, SIMATS Engineering

Presented By:
Mathusri P

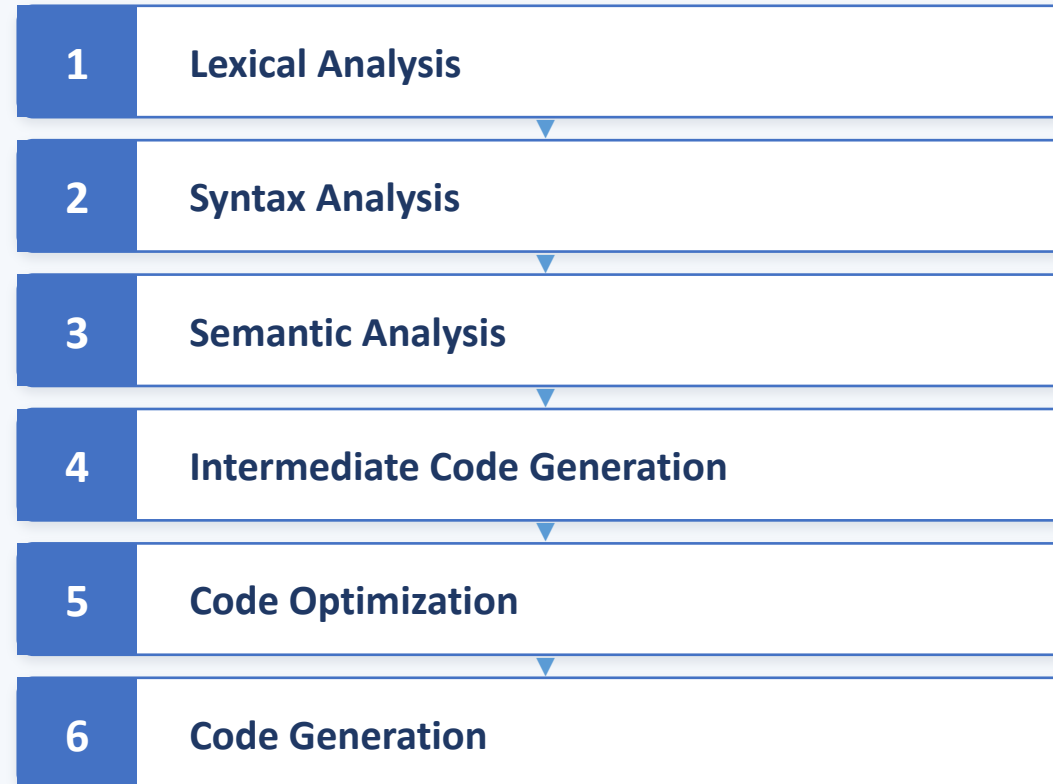
Reg No: 192311363

What is a Compiler?

- A compiler is a program that translates source code written in a high-level language into target code (machine or assembly language).
- It works phase by phase, with each phase transforming the program from one representation to the next.
- The process is broadly divided into two parts: the Analysis phase (front end) and the Synthesis phase (back end).
- A Symbol Table and Error Handler operate alongside every phase, tracking identifiers and reporting mistakes.

The Six Phases

Source code passes through these stages to become executable target code



Phase 1: Lexical Analysis

Also called Scanning

- Reads the source program character by character and groups characters into meaningful sequences called lexemes.
- Produces a stream of tokens, each represented as <token-type, attribute-value>.
- Removes whitespace, comments, and unnecessary characters from the source code.
- Detects lexical errors such as illegal characters or malformed tokens.
- Example: the statement `x = a + b * 2` becomes tokens `id(x)`, `=`, `id(a)`, `+`, `id(b)`, `*`, `num(2)`.

Phase 2: Syntax Analysis

Also called Parsing

- Takes the stream of tokens from the lexical analyzer and checks whether they form a valid syntactic structure.
- Builds a hierarchical tree structure, usually a Parse Tree or Abstract Syntax Tree (AST).
- Verifies the grammar of the source language using context-free grammar rules.
- Reports syntax errors, such as missing semicolons or unbalanced parentheses.
- Common techniques: Top-Down Parsing (LL) and Bottom-Up Parsing (LR).

Phase 3: Semantic Analysis

- Checks the source program for semantic consistency using information from the syntax tree and symbol table.
- Performs type checking, ensuring operators and operands match in type.
- Verifies variables are declared before use and function calls match their definitions.
- Produces an annotated (decorated) syntax tree carrying type and scope information forward.
- Reports semantic errors, such as type mismatches or undeclared variables.

Phase 4: Intermediate Code Generation

- Converts the annotated syntax tree into an Intermediate Representation (IR) of the source program.
- IR is machine-independent, making the compiler easier to retarget to different platforms.
- Common forms include Three-Address Code, Postfix Notation, and Syntax Trees.
- Acts as a bridge between the front end (analysis) and back end (synthesis) of the compiler.
- Example: $a = b + c * d$ may become $t1 = c * d; a = b + t1$.

Phase 5: Code Optimization

- Improves the intermediate code so that the resulting target program runs faster and uses fewer resources.
- Eliminates redundant computations, unreachable code, and unnecessary variables.
- Techniques include constant folding, common sub-expression elimination, and loop optimization.
- Optimization can be local (within a block), global (across a function), or machine-dependent.
- The semantic meaning of the program must remain unchanged after optimization.

Phase 6: Code Generation

The final phase

- Translates the optimized intermediate code into the target language, typically machine or assembly code.
- Performs memory management, assigning storage locations to variables.
- Handles register allocation and instruction selection for the target machine.
- Produces relocatable machine code that can be executed or linked into an executable file.

Supporting Components

Symbol Table and Error Handler work across all phases

- Symbol Table: a data structure that stores information about identifiers - names, types, scopes, and memory locations.
- Every phase, from lexical analysis to code generation, reads from or updates the symbol table.
- Error Handler: detects and reports errors encountered at any phase of compilation.
- Good error handling helps the compiler recover and continue, reporting multiple errors in a single pass.

Conclusion

- A compiler transforms high-level source code into target code through six well-defined phases.
- Each phase performs a specific transformation: from tokens, to trees, to intermediate code, to optimized machine code.
- The Symbol Table and Error Handler support every phase throughout the process.
- Understanding these phases is fundamental to understanding how programming languages are implemented.

Thank You

Mathusri P | Reg No: 192311363

CSA1412 - Compiler Design | Dept. of CSE, SIMATS Engineering